

Development Plan Software Engineering

Team 13, Keystone

Ayush Patel
Ahmed Rashrash
Andrew Habib
Waqar Ul-Hassan

Table 1: Revision History

Date	Developer(s)	Change
September 25th 2026	Ayush Patel	Added intro and Sections 1 - 3
September 26th 2026	All Team Members	Added Team Meeting and Communication Plan and Team Roles
September 26th 2026	All Team Members	Added Expected Technology and AI Use and Team Charter
Sept 26th 2026	Waqar Ul-Hassan	Added reflection information
September 26th 2026	Andrew Habib	Added Coding Standard Section 12
...	...	...

This document contains details about the development plan for the project. It includes information about the confidentiality and licensing (sections 1 - 3), team communication and meeting plans (sections 4 - 6), workflow and project decomposition (sections 7 - 8), proof of concept demonstration plan (section 9), expected technology (section 10), coding standard (section 11), and appendices for generative AI use disclosure, reflection, and team charter.

1 Confidential Information?

There is no confidential information to protect in this project.

2 IP to Protect

The intellectual property produced by this project consists solely of written content, namely the code and documentation, which is covered under our copyright license, discussed below.

3 Copyright License

The copyright license adopted by the team for this project is the MIT license. This license provides others the right to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the software, provided that the copyright notice and permission notice are included in all copies. The license also disclaims warranty and limits the authors' liability for any damages arising from use of the software. The [copyright license](#) for the project can be found in the team repository.

4 Team Meeting Plan

- **Weekly Team Meeting:** Once a week for one hour – Wednesdays, 2:30pm to 3:30pm, in person. Additional virtual meetings will be scheduled as needed when approaching a deliverable deadline.
- **Meeting Chair:** Waqar Ul-Hassan - prepares and sends out the meeting agenda and leads the meetings
- **Notetaker:** Ahmed Rashrash - record the meeting minutes using the template below
- **Supervisor Meetings:** We will meet with Dr. Smith at least once a month, and more frequently as needed. These will primarily occur during lecture time, with virtual meetings as a backup when an in-lecture meeting is not possible.

- **Agenda Requirement:** All meetings (team, TA, supervisor) must have an agenda, prepared and shared with the team at least 24 hours in advance by the chair. The agenda template is as follows:
 - Date, time, attendees
 - Purpose of the meeting
 - Review of previous deliverable/meeting: what went well, what needs improvement
 - Team member updates: completed tasks, outstanding tasks, tasks needing assistance
 - Open questions for supervisor / advisor, TA, or stakeholder (as applicable)
 - Action items and task owners for the next meeting
- **Meeting Minutes Template:** Minutes will be recorded for every team, TA, and supervisor meeting, using the following structure:
 - Date, time, attendees, absentees (with reason if provided)
 - Summary of discussion per agenda item
 - Decisions made
 - Action items, owners, and due dates
 - Open questions carried forward

Minutes will be stored in the description of a "Team Meeting" GitHub Issue created for each meeting, using the provided Team Meeting issue template.

5 Team Communication Plan

- **Primary asynchronous channel:** Microsoft Teams chat, for day-to-day communication and quick questions.
- **Task-level communication:** GitHub – issues, pull requests, and code review comments are used for anything related to a specific task, so discussion stays attached to the relevant work item. Members will mention each other in PRs/issues, assign issues directly, and request reviews through GitHub's review request feature.
- **Meeting minutes:** Recorded and stored on GitHub (see Team Meeting Plan).
- **Supervisor/stakeholder communication:** Email (Dr. Smith prefers email), routed through the team liaison to keep a single point of contact.
- **Response-time expectation:** Each member will respond to direct communication in the team's Teams chat within 24 hours.

6 Team Member Roles

- **Project-Level Roles:**

- Team Liaison: Ayush Patel – primary point of contact with the supervisor and course staff.
- Meeting Chair: Waqar Ul-Hassan – prepares and sends out the meeting agenda and leads the meetings.
- Notetaker: Ahmed Rashrash – record the meeting minutes using the template below.
- GitHub Manager: Andrew Habib – responsible for keeping labels, issue templates, branch protection rules, and the project board consistent and up to date.

- **Documentation Quality Assurance:** Rotates per deliverable alphabetically by first name starting with Ahmed for the first deliverable. The member(s) in this role are responsible for checking the completed deliverable against both the repository checklist and the Avenue rubric before submission, and flagging any gaps to the team with enough time to address them before the deadline.

- This is in addition to, not a replacement for, normal PR review.

- **Technical Roles:**

- Frontend developer: Responsible for development of a user friendly interface - All members
- Backend developer: Responsible for developing the backend and database layer - All members
- Reviewer (code): all members.
- Reviewer (documentation): all members.
- Technical QA: Responsible for doing functional testing after each code functionality related PR is merged. This will rotate for each deliverable and will be the same team member who is also the documentation QA.

We are intentionally keeping technical roles broad and overlapping at this stage rather than assigning rigid specializations, since the team is still early in the project and all members should be able to work across the stack. Specializations may emerge naturally as the project progresses.

- **Flexibility and Role Changes:** Roles are not fixed for the duration of the project. At the start of each milestone, the team will briefly revisit role assignments based on interest, workload balance, and skills members want to develop, and reassign as needed. This ensures no single member is locked into a role that no longer fits the project's needs or their own goals.

- **Overlap by Design:** Technical and review roles are deliberately assigned to "all" members rather than single individuals, so there is no single point of failure if one member is unavailable during a critical period.

7 Workflow Plan

- How will you be using git, including branches, pull request, etc.?
- How will you be managing issues, including template issues, issue classification, etc.?
- Use of CI/CD

8 Project Decomposition and Scheduling

GitHub Projects

GitHub projects will be the primary place where issues are tracked. Each issue is part of a milestone, which is a group of issues that have an explicit due date. Issues move through the stages using a Kanban-style board in the following order: Backlog → Ready → In-Progress → In-Review → Done.

Each issue has a title, a description if needed, a set of assignees, priority level, and classification (feature, bug, task, documentation, meeting). Milestone due dates correspond to deliverable deadlines.

[Link to GitHub Project](#)

Schedule

This project is organized into 10 milestones spanning two terms.

- Milestone 1 (Weeks 3-4): Problem Statement, Proof of Concept Plan, Development Plan, Team Charter
- Milestone 2 (Weeks 5-6): Software Requirements Specification, Hazard Analysis (Rev 0), User/Stakeholder/Client Interview Report (Fall Custom Document Rev 0)
- Milestone 3 (Weeks 7-8): Verification and Validation Plan (Rev 0)
- Milestone 4 (Weeks 9-10): Design Document (Rev -1)
- Milestone 5 (Weeks 11-13): Proof of Concept Demo, Interviews
- Milestone 6 (Weeks 14-16): Design Document (Rev 0)
- Milestone 7 (Weeks 17-19): Revision 0 Demo, Interviews

- Milestone 8 (Weeks 20-22): Verification and Validation Report (Rev 0), Usability Testing Report (Winter Custom Document Rev 0)
- Milestone 9 (Weeks 23-24): Final Demonstration (Rev 1)
- Milestone 10 (Weeks 25-26): EXPO Demonstration, Final Documentation (Rev 1)

9 Proof of Concept Demonstration Plan

What is the main risk, or risks, for the success of your project? What will you demonstrate during your proof of concept demonstration to convince yourself that you will be able to overcome this risk?

10 Expected Technology

- **Version Control and Project Management:** Git, Github and GitHub projects
- **Frontend:** React – chosen for its large ecosystem, team familiarity, and strong component-based structure suited to an interactive web application. **Trade-off:** steeper initial setup compared to simpler frameworks, but this is offset by better long-term maintainability.
- **Backend:** Python with FastAPI – chosen for fast development speed, built-in data validation, automatic API documentation, and strong support for any data-processing or ML-adjacent work the project may require. **Trade-off:** lower raw performance than a compiled-language backend, which is an acceptable trade-off given the project’s expected scale.
- **Backend unit testing:** pytest – chosen over the built-in `unittest` module for its more concise syntax, fixture system, and better plugin ecosystem for coverage reporting.
- **Frontend unit testing:** Jest – chosen for its tight integration with React and minimal configuration overhead.
- **E2E Testing:** Playwright – industry standard for end to end testing of web applications
- **Frontend linter:** ESLint – industry standard for TypeScript/JavaScript, configurable to enforce team-agreed style rules.
- **Backend linter:** Ruff – industry standard linter, easiest to setup, fastest and most modern
- **Unit test coverage tool:** SonarQube – industry standard for reporting and visualizing code coverage

- **Database:** PostgreSQL - relational database will make it easier
- **Application Packaging:** Docker - production standard to package application components and dependencies into one executable container **Trade-off:** Extra set up is required but this will ensure our code will run anywhere we choose to deploy in the future
- **Continuous Integration:** GitHub Actions. CI will run on every push and pull request, checking: linting (ESLint/Pylint), unit test pass/fail (Jest/pytest), and SonarQube coverage reports.
- *Note: the technologies listed here reflect the team's current expectations and are not binding constraints. They may evolve as the project's design becomes more concrete.*

11 Use of AI and LLMs

- **Purpose:** AI will be used to accelerate development and ensure deliverables align with the rubric and checklists, while the team retains ownership of all thinking, design, and logic decisions.
- **Tools:** Claude and ChatGPT for documentation and code implementation assistance, GitHub Copilot or Claude Code for in-editor code assistance.
- **Documentation use of AI:**
 - Brainstorming and idea generation
 - Proofreading, grammar, and spelling checks.
 - Assistance with LaTeX syntax and compilation issues.
 - Checking documents against the rubric and checklist for completeness.

Documentation content itself will be written by the team, not generated wholesale by AI.

- **Code use of AI:**
 - A walking skeleton of the codebase must be written manually by the team.
 - Function logic should first be written out as pseudocode by a team member; AI may then be used to help implement that pseudocode, rather than generating logic from scratch.
 - AI is not used to make architecture or design decisions – these remain human decisions.

- All AI-generated code must be reviewed line by line before being committed. Data structure choices, naming conventions, and other design decisions are made by the team, not the AI.
 - AI may assist in generating unit tests and identifying edge cases, but all generated tests must be reviewed and understood by the team, not accepted blindly.
 - Team members must manually verify that AI-generated tests are actually testing the intended behaviour (e.g. by intentionally breaking the implementation and confirming the test fails).
- *Note: this section covers AI's role in building the project. AI use in drafting this document itself is disclosed separately in the Generative AI Use Disclosure appendix.*

12 Coding Standard

The team will follow the coding standards and conventions below to ensure consistency, readability, maintainability, and code quality across the project:

- **Python / FastAPI:** Python code should adhere to **PEP 8**. Ruff is the Python Linter used to enforce these standards and identify common code-quality issues.
- **Typescript / React:** Frontend code will follow consistent Typescript and React conventions for naming, formatting, component structure, and static typing typically enforced by ESLint.
- **Software Design/Architectural Principles: SOLID and GRASP** principles should be used to ensure proper design principles are followed such as single responsibility, separation of concerns, high cohesion, low coupling, modularity, abstraction, and extensibility. Following these will allow the project to have a maintainable system design.
- **Testing Standards:** Test design should be guided by ISO/IEC/IEEE 29119-4 to ensure consistent software testing practices based on requirements. The standards cover software testing techniques used to evaluate boundary conditions, edge cases, and expected failure conditions.
- **Statement Coverage:** Automated tests should target approximately 80% statement coverage for application code. Statement coverage along with test design standards defined above should determine overall desired test quality.
- **Readability and Naming:** Code should use descriptive and consistent names and favour clear, simple implementations over unnecessary complexity. Comments should give high-level implementation explanations.

- **Error Handling:** Failure conditions should be handled explicitly, with clear and consistent error messages.

13 Appendix — Generative AI Use Disclosure

13.1 AI Tools Used

13.2 How and Where Used

13.3 Verification of AI-Generated Content

Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

Waqar Ul-Hassan:

1. Why is it important to create a development plan prior to starting the project?

For a project of this size, especially because we are developing it from scratch it is critical that we create a plan to ensure we have a scope of the project. We can set realistic expectations and everyone in the group is on the same page to ensure we don't have any miscommunication. Additionally, this way we are also able to identify risks earlier on by evaluating the project and thinking about it from a more management perspective.

2. In your opinion, what are the advantages and disadvantages of using CI/CD?

I feel that the biggest advantage of using CI/CD is to have an automated testing suite which will build when you push code and catch bugs, code formatting, etc. This ensure that we are keeping a certain standard of work in our active repository and overall are more confident when pushing to the main branch. The biggest disadvantage I think is definitely the maintenance of the CI/CD pipeline, making sure its up to date. On top of this if the pipeline is not made very well, it can cause some very frustrating problems and become a huge waste of time, especially when trying to scale a project.

3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Initially we had some disagreements on things like, which backend language we should use with our react frontend. One of my group members wanted to use nextJS while others wanted to use python FastAPI. Another, disagreement we had was when deciding on the metric to use to check for contribution amount to the project. Some wanted to use lines of code, commits, and others wanted to use number of issues. For all these initial disagreements, we just held discussions during our meetings.

In these discussions, we weighed the pros and cons of using each of the different methodologies. After weighing them together, we then decided for example to use python FastAPI as we feel we have more experience with it and also its much more flexible as well as use the number of issues as our comparison metric.

Appendix — Team Charter

External Goals

- Produce something we are proud of and can discuss meaningfully in interviews.
- Given the nature of the project, aim for it to be genuinely useful to future capstone students and course staff after we graduate.
- Improve our project management, design, and technical skills by applying them to a real-world problem.
- Get an A+

Attendance

Expectations

- 80% attendance is required for team, TA, and supervisor meetings.
- If a member must miss a meeting, they must notify the team in advance and provide a written update covering their progress and any action items relevant to them.
- Members attending a meeting are expected to arrive on time (at most 10 minutes late) and stay for its full duration.
- **Trigger:** Missing 2 consecutive meetings, *or* being late thrice in a row, triggers a consequence.
- **Consequence:** the member brings snacks/coffee to the next team meeting for a first occurrence; a second occurrence within the same term triggers a mandatory check-in meeting with the TA to discuss the pattern.
- These standards apply to non-emergency cases only (see below for emergencies).

Acceptable Excuse

- **Acceptable:** midterm preparation, conflicting lectures/tutorials, work-related obligations, medical appointments.
- **Not acceptable:** forgetting the meeting, non-urgent personal plans, or repeated last-minute excuses of the same kind. Repeated use of the same excuse will be treated as a pattern and addressed under the "Stay on Track" consequences below.
- Advance notice is expected wherever the excuse is foreseeable (e.g. a scheduled appointment).

In Case of Emergency

- In a genuine emergency, the attendance rules above do not apply.
- The member should notify the team as soon as reasonably possible.
- If the emergency will materially impact a deliverable, there are two options:
 1. The other members will redistribute the member's tasks among themselves, and then that team member can pick up more work in future deliverables.
 2. If the rest of the team cannot handle the extra workload, the team will raise it with the TA and/or course instructor proactively, rather than waiting until the deadline is missed.

Accountability and Teamwork

Quality

- Deliverables must follow the AI usage guidelines in this document – no unreviewed "AI slop."
- Every deliverable is checked against both the Avenue rubric and the repository checklist before submission, by the assigned documentation QA role for that deliverable (see Team Member Roles) in addition to the author.
- Members are expected to arrive at meetings ready to give a meaningful update or raise questions.
- Members should stay reasonably current on lecture content and upcoming deliverable requirements.

Attitude

- Team members are expected to be positive, respectful, and willing to work hard.
- All members' ideas are given fair consideration; disagreement is handled respectfully, not dismissively.
- Members are expected to communicate openly rather than let frustration build silently.

Stay on Track

- Progress is tracked via the GitHub Project board – issue status, assignee, and milestone fields are kept current so contribution is visible to the whole team, not just self-reported.

- **Regular Contributions:** Each member should have visible, regular activity (commits, PRs, or issue updates) across each week of an active deliverable period, not concentrated entirely in the final 1–2 days.
- **Target metrics:** Issues will be assigned at the beginning of each deliverable or during each weekly meeting. We will try to assign each person a relatively equal amount of work and finishing those issues within the provided deadline will be the main target metric.
- **Consequence for falling behind and repeated excuses:** First instance – a direct, informal conversation with the member to understand blockers and rebalance tasks; a repeated pattern across multiple deliverables will result in a joint conversation involving the TA.

Team Building

We will play ping pong together.

Decision Making

- Decisions are made by consensus where possible.
- Where consensus cannot be reached, the team will vote; the majority decision stands.
- Where a disagreement cannot be resolved by voting, the team will default to whichever option best aligns with the rubric and industry best practice; if still unresolved, the TA or supervisor will be brought in to mediate.